Ejercicio 1

Quiero implementar en Java 21 una clase llamada PilaEnteros que funcione con un arreglo de tamaño fijo recibido por parametro en el constructor. La pila debe tener las operaciones basicas: push(), pop(), peek(), isEmpty(), isFull() y size(). Se debe incluir un indice top para controlar el tope de la estructura, el cual arranca inicialmente en -1 porque la posicion 0 indicaria que ya hay un dato cargado. Hay que contemplar los casos limite: cuando se intente hacer push estando la pila llena debe lanzar error o excepcion por overflow, y al hacer pop o peek con la pila vacia debe controlar el underflow. Se debe probar insertando elementos hasta llenarla, consultando el tope y desapilando todo hasta vaciarla.

Caso 1 (Inserción y consulta normal): Crear pila con capacidad 3. Hacer push(10) y push(20). Probar peek() → debe devolver 20. Probar size() → debe devolver 2.

Caso 2 (Desbordamiento / Overflow): Con la pila anterior, hacer push(30) (isFull() pasa a ser true). Intentar un cuarto push(40) → debe lanzar excepción o mensaje de error indicando que la pila está llena (Overflow).

Caso 3 (Vaciado y Underflow): Ejecutar pop() tres veces consecutivas. Verificar que devuelva los valores en orden inverso (30, 20, 10). Comprobar que isEmpty() sea true y top == -1. Intentar un nuevo pop() → debe lanzar error o excepción de Pila Vacía (Underflow).

```
public class PilaEnteros {
    private int[] elementos;
    private int top;

    // Constructor: recibe la capacidad de la pila
    public PilaEnteros(int capacidad) {
        if (capacidad <= 0) {
            throw new IllegalArgumentException(
                "La capacidad debe ser mayor que cero."
            );
        }
        elementos = new int[capacidad];
        top = -1;
    }
```

```
// Inserta un elemento en el tope
public void push(int valor) {
    if (isFull()) {
        throw new IllegalStateException(
            "Overflow: la pila está llena."
        );
    }

    top++;
    elementos[top] = valor;
}

// Elimina y devuelve el elemento del tope
public int pop() {
    if (isEmpty()) {
        throw new IllegalStateException(
            "Underflow: la pila está vacía."
        );
    }

    int valor = elementos[top];
    top--;

    return valor;
}

// Consulta el elemento del tope sin eliminarlo
public int peek() {
```

```
        if (isEmpty()) {
            throw new IllegalStateException(
                "Underflow: la pila está vacía."
            );
        }
        return elementos[top];
    }
    // Comprueba si la pila está vacía
    public boolean isEmpty() {
        return top == -1;
    }
    // Comprueba si la pila está llena
    public boolean isFull() {
        return top == elementos.length - 1;
    }
    // Devuelve la cantidad de elementos
    public int size() {
        return top + 1;
    }
}
```

```
=== CASO 1 ===
Tope: 20
Tamaño: 2

=== CASO 2 ===
Pila llena: true
Overflow: la pila está llena.
```

=== CASO 3 ===

Pop: 30

Pop: 20

Pop: 10

Pila vacía: true

Tamaño: 0

Underflow: la pila está vacía.

¿Está vacía?: true

Ejercicio 2

Necesito implementar en Java 21 un programa que simule una torre de platos donde cada plato se represente con un nombre o numero. El programa debe permitir tres acciones principales: agregar un plato arriba de todo, retirar el plato superior y consultar cual es el plato que quedo arriba de todo sin sacarlo. Este problema se debe implementar con una pila porque es una estructura LIFO (ultimo en entrar, primero en salir), ya que el ultimo plato apoyado arriba es fisicamente el primero que se tiene que retirar para no romper los demas. El sistema debe usar internamente un arreglo fijo, manejar errores si la torre se llena o si intentan retirar un plato cuando no queda ninguno, y tener una prueba en consola mostrando como se van apilando y retirando platos.

Caso 1 (Apilado sucesivo): Apilar en orden: "Plato Blanco", "Plato Azul", "Plato Rojo". Consultar el plato superior con peek → debe mostrar "Plato Rojo".

Caso 2 (Principio LIFO): Retirar un plato → debe salir "Plato Rojo". Volver a consultar la cima → debe quedar "Plato Azul".

Caso 3 (Torre vacía): Retirar los dos platos restantes ("Plato Azul" y "Plato Blanco"). Intentar retirar un plato adicional → el sistema debe avisar que no quedan platos en la torre para retirar.

```
public class TorrePlatos {


    private String[] platos;

    private int top;
```

```
// Constructor
public TorrePlatos(int capacidad) {
    if (capacidad <= 0) {
        throw new IllegalArgumentException(
            "La capacidad debe ser mayor que cero."
        );
    }

    platos = new String[capacidad];
    top = -1;
}

// Agregar un plato arriba de todo
public void push(String plato) {
    if (isFull()) {
        throw new IllegalStateException(
            "Overflow: la torre está llena."
        );
    }

    top++;
    platos[top] = plato;
}

// Retirar y devolver el plato superior
public String pop() {
    if (isEmpty()) {
        throw new IllegalStateException(
```

```
            "Underflow: no quedan platos en la torre para retirar."
        );
    }

    String plato = platos[top];
    platos[top] = null;
    top--;

    return plato;
}

// Consultar el plato superior sin retirarlo
public String peek() {
    if (isEmpty()) {
        throw new IllegalStateException(
            "Underflow: la torre está vacía."
        );
    }

    return platos[top];
}

// Verificar si la torre está vacía
public boolean isEmpty() {
    return top == -1;
}

// Verificar si la torre está llena
```

```
    public boolean isFull() {
        return top == platos.length - 1;
    }

    // Obtener la cantidad de platos
    public int size() {
        return top + 1;
    }
}
```

```
=== CASO 1: APILADO ===
Se agregó: Plato Blanco
Se agregó: Plato Azul
Se agregó: Plato Rojo
Plato superior: Plato Rojo
Cantidad de platos: 3

=== CASO 2: RETIRO LIFO ===
Se retiró: Plato Rojo
Nuevo plato superior: Plato Azul

=== CASO 3: TORRE VACÍA ===
Se retiró: Plato Azul
Se retiró: Plato Blanco
¿La torre está vacía?: true
Cantidad de platos: 0
Underflow: no quedan platos en la torre para retirar.
```

Ejercicio 3

Quiero hacer un programa en Java 21 que reciba una expresion matematica en un string y verifique si los parentesis estan correctamente balanceados devolviendo valido o invalido. Se debe utilizar una pila porque cada vez que el algoritmo recorre la cadena y encuentra un parentesis de apertura '(' lo guarda haciendo push en la pila, y cuando encuentra uno de cierre ')' hace un pop para emparejarlo. Si al querer hacer pop la pila esta vacia, o si al terminar de leer todo el texto la pila todavia tiene parentesis adentro que sobraron, la expresion es invalida; solo es valida si termina exactamente vacia (isEmpty). Debe manejar casos extremos como cadenas vacias o sin parentesis. Quiero probarlo con:

- (5 + 3) * (2 + 1) (debe dar valido)
- (5 + 3)) * (2 + 1 (debe dar invalido)
- ((2 * 3) (debe dar invalido)
-) (debe dar invalido)

Ejercicio 4

Se requiere un programa en Java 21 que simule el historial de un navegador web para navegar entre URLs. Cada vez que el usuario ingresa o visita una nueva pagina web, la direccion URL se apila arriba de todo usando push. El programa debe permitir volver a la pagina anterior mediante la operacion pop, mostrando a cual pagina regreso. Un historial funciona solamente bajo el principio LIFO porque la ultima pagina web que abriste es justamente la primera que necesitas ver cuando apretas el boton atras. Hay que contemplar el caso limite cuando no hay mas paginas a las cuales volver atras avisando que el historial esta vacio. El codigo debe estar comentado y mostrar una ejecucion navegando por 3 o 4 paginas y volviendo atras hasta el inicio.

Caso 1 (Navegación secuencial): Visitar secuencialmente: "google.com", "github.com", "stackoverflow.com".

Caso 2 (Retroceso / Botón Atrás): Presionar "Atrás" (ejecutar pop()) → debe retirar "stackoverflow.com" y dejar al usuario posicionado en "github.com".

Caso 3 (Retroceso hasta el límite): Presionar "Atrás" dos veces más hasta salir de "google.com". Presionar "Atrás" una vez más → el sistema debe indicar que no hay más páginas previas en el historial.

```
public class ValidadorParentesis {


    private char[] pila;

    private int top;
```

```
// Constructor de la pila
public ValidadorParentesis(int capacidad) {
    pila = new char[capacidad];
    top = -1;
}

// Agregar un caracter a la pila
public void push(char caracter) {
    if (top == pila.length - 1) {
        throw new IllegalStateException("Overflow: pila llena.");
    }

    top++;
    pila[top] = caracter;
}

// Sacar un caracter de la pila
public char pop() {
    if (isEmpty()) {
        throw new IllegalStateException("Underflow: pila vacía.");
    }

    char caracter = pila[top];
    top--;

    return caracter;
```

```
}

// Verificar si la pila está vacía
public boolean isEmpty() {
    return top == -1;
}

// Verificar si los paréntesis están balanceados
public boolean esValida(String expresion) {

    // Reiniciar la pila para validar una nueva expresión
    top = -1;

    for (int i = 0; i < expresion.length(); i++) {

        char caracter = expresion.charAt(i);

        if (caracter == '(') {
            push(caracter);
        }

        else if (caracter == ')') {

            // No puede cerrar si no hay una apertura previa
            if (isEmpty()) {
                return false;
```

```
            }

            pop();
          }
        }

        // Es válida únicamente si no quedaron aperturas pendientes
        return isEmpty();
    }
}
```

Expresión: "(5 + 3) * (2 + 1)"

Resultado: VÁLIDO

Expresión: "(5 + 3)) * (2 + 1"

Resultado: INVÁLIDO

Expresión: "((2 * 3)"

Resultado: INVÁLIDO

Expresión: ")"

Resultado: INVÁLIDO

Expresión: ""

Resultado: VÁLIDO

Expresión: "5 + 3 * 2"

Resultado: VÁLIDO

Ejercicio 5

Quiero implementar en Java 21 una clase Pila<T> utilizando arreglos para que pueda almacenar cualquier tipo de objeto generico como Integer, String u objetos simples. Los tipos genericos permiten crear estructuras de datos reutilizables y con tipado seguro en tiempo de compilacion, evitando tener que programar una clase distinta para cada tipo (como PilaEnteros, PilaStrings, etc.) o hacer casteos con Object. El codigo debe estar comentado y validarse con una prueba usando Strings y otra con numeros enteros.

Caso 1 (Prueba con Integer): Instanciar Pila<Integer> de capacidad 3. Insertar 100, 200 y 300. Hacer pop() → debe retornar el número entero 300.

Caso 2 (Prueba con String): Instanciar Pila<String> de capacidad 2. Insertar "Primero", "Segundo". Probar peek() → debe retornar "Segundo" sin necesidad de realizar casteos manuales ((String)).

Caso 3 (Prueba con objeto personalizado): Instanciar Pila<Usuario> o Pila<Persona> con un objeto propio y verificar que apile y desapile conservando los atributos del objeto.

```
public class Pila<T> {

    // Arreglo que almacena los elementos de la pila.
    // Java no permite crear directamente un arreglo de tipo T[],
    // por eso internamente usamos un arreglo de Object.
    private Object[] elementos;

    // Índice del último elemento insertado.
    // -1 significa que la pila está vacía.
    private int top;

    // Constructor: recibe la capacidad fija de la pila.
```

```
public Pila(int capacidad) {
    if (capacidad <= 0) {
        throw new IllegalArgumentException(
            "La capacidad debe ser mayor que cero."
        );
    }

    elementos = new Object[capacidad];
    top = -1;
}

// Inserta un elemento en el tope de la pila.
public void push(T elemento) {
    if (isFull()) {
        throw new IllegalStateException(
            "Overflow: la pila está llena."
        );
    }

    top++;
    elementos[top] = elemento;
}

// Elimina y devuelve el elemento que está en el tope.
public T pop() {
    if (isEmpty()) {
```

```
            throw new IllegalStateException(
                "Underflow: la pila está vacía."
            );
        }

        // Recuperamos el elemento del tope.
        @SuppressWarnings("unchecked")
        T elemento = (T) elementos[top];

        // Liberamos la referencia y actualizamos el tope.
        elementos[top] = null;
        top--;

        return elemento;
    }

    // Devuelve el elemento superior sin eliminarlo.
    public T peek() {
        if (isEmpty()) {
            throw new IllegalStateException(
                "Underflow: la pila está vacía."
            );
        }

        @SuppressWarnings("unchecked")
        T elemento = (T) elementos[top];
```

```
        return elemento;
    }

    // Indica si la pila está vacía.
    public boolean isEmpty() {
        return top == -1;
    }

    // Indica si la pila alcanzó su capacidad máxima.
    public boolean isFull() {
        return top == elementos.length - 1;
    }

    // Devuelve la cantidad de elementos almacenados.
    public int size() {
        return top + 1;
    }
}
=== CASO 1: INTEGER ===
Número retirado: 300
Tamaño restante: 2

=== CASO 2: STRING ===
Tope de la pila: Segundo
Tamaño: 2
```

=== CASO 3: USUARIO ===

Usuario retirado: Usuario{nombre='Lucas', edad=25}

Nombre: Lucas

Edad: 25

Nuevo tope: Usuario{nombre='Ana', edad=20}

Ejercicio 6

Necesito implementar en Java 21 una clase llamada ColaEnteros utilizando un arreglo. Debe permitir realizar las siguientes operaciones: enqueue(), dequeue(), front(), isEmpty(), isFull() y size(). El indice front indica la posicion del primer elemento listo para salir y avanza cuando se desencola con dequeue, mientras que rear indica el final de la fila y avanza en cada insercion con enqueue. El codigo debe validar los casos limite de overflow y underflow. Se debe incluir una prueba donde se encolen 4 numeros, se consulte el frente y se vayan sacando de a uno respetando el orden de llegada.

Caso 1 (Encolar y respetar FIFO): Crear cola de tamaño 4. Hacer enqueue(10), enqueue(20), enqueue(30). Consultar front() → debe retornar 10.

Caso 2 (Desencolar): Ejecutar dequeue() → debe retornar 10. Consultar nuevamente front() → ahora debe ser 20 (demuestra que el frente avanzó).

Caso 3 (Casos límite): Desencolar los elementos restantes hasta vaciarla (isEmpty() == true). Intentar hacer dequeue() → debe lanzar error de cola vacía (Underflow).

```
public class ColaEnteros {

    private int[] elementos;

    private int front;

    private int rear;

    private int cantidad;

    // Constructor: crea una cola con capacidad fija.
```

```
public ColaEnteros(int capacidad) {
    if (capacidad <= 0) {
        throw new IllegalArgumentException(
            "La capacidad debe ser mayor que cero."
        );
    }

    elementos = new int[capacidad];
    front = 0;
    rear = -1;
    cantidad = 0;
}

// Agrega un número al final de la cola.
public void enqueue(int valor) {
    if (isFull()) {
        throw new IllegalStateException(
            "Overflow: la cola está llena."
        );
    }

    rear++;
    elementos[rear] = valor;
    cantidad++;
}
```

```
// Elimina y devuelve el primer número de la cola.
public int dequeue() {
    if (isEmpty()) {
        throw new IllegalStateException(
            "Underflow: la cola está vacía."
        );
    }

    int valor = elementos[front];

    // Liberamos la posición que acabamos de utilizar.
    elementos[front] = 0;

    // Avanzamos al siguiente elemento.
    front++;
    cantidad--;

    return valor;
}

// Consulta el primer elemento sin eliminarlo.
public int front() {
    if (isEmpty()) {
        throw new IllegalStateException(
            "Underflow: la cola está vacía."
        );
```

```
        }

        return elementos[front];
    }

    // Indica si no quedan elementos.
    public boolean isEmpty() {
        return cantidad == 0;
    }

    // Indica si no quedan posiciones disponibles al final.
    public boolean isFull() {
        return rear == elementos.length - 1;
    }

    // Devuelve la cantidad actual de elementos.
    public int size() {
        return cantidad;
    }
}
```

```
=== CASO 1: ENCOLAR ===
Frente: 10
Tamaño: 4
¿Está llena?: true
Overflow: la cola está llena.
```

=== CASO 2: DESENCOLAR ===

Sale: 10

Nuevo frente: 20

Tamaño: 3

=== CASO 3: VACIAR COLA ===

Sale: 20

Sale: 30

Sale: 40

¿Está vacía?: true

Tamaño: 0

Underflow: la cola está vacía.

Ejercicio 7

Quiero implementar en Java 21 un sistema para una oficina donde las personas van llegando, sacan un turno y se van anotando en una cola. El programa debe permitir tres operaciones basicas: agregar a una persona al final de la fila, llamar y atender a la siguiente persona liberando su lugar, y consultar quien es el proximo en la fila sin atenderlo todavia. Este sistema se debe implementar con una estructura FIFO y no con LIFO, ya que la primer persona en llegar a la sala de espera debe ser estrictamente la primera en ser atendida. Se deben manejar excepciones si no hay personas en espera y mostrar por consola cuando se agregan y se llaman personas.

Caso 1 (Llegada de clientes): Encolar a "Turno 1 - Juan", "Turno 2 - Maria", "Turno 3 - Carlos". Consultar el próximo turno sin atender → debe devolver "Turno 1 - Juan".

Caso 2 (Atención en orden estricto FIFO): Llamar a atender → debe salir "Turno 1 - Juan". Volver a llamar → debe salir "Turno 2 - Maria".

Caso 3 (Oficina sin clientes en espera): Atender a "Turno 3 - Carlos". Intentar atender a una persona más → el sistema debe mostrar un mensaje avisando que la sala de espera está vacía.

```
public class ColaTurnos {

    private String[] turnos;
    private int front;
    private int rear;
    private int cantidad;

    // Constructor: establece la capacidad de la cola.
    public ColaTurnos(int capacidad) {
        if (capacidad <= 0) {
            throw new IllegalArgumentException(
                "La capacidad debe ser mayor que cero."
            );
        }

        turnos = new String[capacidad];
        front = 0;
        rear = -1;
        cantidad = 0;
    }

    // Agrega una persona al final de la fila.
    public void enqueue(String turno) {
        if (isFull()) {
            throw new IllegalStateException(
```

```
            "Overflow: la sala de espera está llena."
        );
    }

    rear++;
    turnos[rear] = turno;
    cantidad++;

    System.out.println("Llegó una persona: " + turno);
}

// Atiende y elimina a la primera persona de la fila.
public String dequeue() {
    if (isEmpty()) {
        throw new IllegalStateException(
            "Underflow: la sala de espera está vacía."
        );
    }

    String turno = turnos[front];

    // Liberar la posición ocupada.
    turnos[front] = null;

    // Avanzar el frente y actualizar la cantidad.
    front++;
```

```
    cantidad--;

    System.out.println("Se llama a atender: " + turno);

    return turno;
}

// Consulta el próximo turno sin atenderlo.
public String front() {
    if (isEmpty()) {
        throw new IllegalStateException(
            "La sala de espera está vacía."
        );
    }

    return turnos[front];
}

// Comprueba si no hay personas esperando.
public boolean isEmpty() {
    return cantidad == 0;
}

// Comprueba si se alcanzó el final del arreglo.
public boolean isFull() {
    return rear == turnos.length - 1;
```

```
    }

    // Devuelve la cantidad de personas en espera.
    public int size() {
        return cantidad;
    }
}
```

```
=== CASO 1: LLEGADA DE CLIENTES ===
Llegó una persona: Turno 1 - Juan
Llegó una persona: Turno 2 - Maria
Llegó una persona: Turno 3 - Carlos
Próximo turno: Turno 1 - Juan
Personas en espera: 3

=== CASO 2: ATENCION ===
Se llama a atender: Turno 1 - Juan
Se llama a atender: Turno 2 - Maria
Próximo turno: Turno 3 - Carlos

=== CASO 3: SALA VACIA ===
Se llama a atender: Turno 3 - Carlos
¿Sala vacía?: true
Underflow: la sala de espera está vacía.
```

Ejercicio 8

Se requiere implementar en Java 21 un simulador de cola de impresion para una oficina utilizando un arreglo fijo. Cada documento a imprimir tiene un nombre y una cantidad de paginas cargadas. Los archivos que manda cada usuario deben salir impresos exactamente en el mismo orden en que fueron enviados. Se debe implementar la estructura FIFO para hacer la cola de impresion ya que el primer archivo mandado se va a imprimir antes que los que llegaron mas tarde. El programa debe soportar mandar documentos a imprimir, procesar la impresion del siguiente documento retirandolo y mostrar el documento que esta esperando en el frente.

Caso 1 (Envío de trabajos de impresión): Agregar a la cola tres documentos:

- Doc1: ("Tesis.pdf", 50 páginas)
- Doc2: ("Contrato.docx", 5 páginas)
- Doc3: ("Foto.png", 1 página)

Caso 2 (Procesamiento secuencial): Imprimir siguiente documento → debe imprimir "Tesis.pdf" (a pesar de tener muchas más páginas, se atiende primero por orden de llegada).

Caso 3 (Verificación del siguiente en espera): Consultar el frente de la cola de impresión → debe indicar que el próximo documento en fila es "Contrato.docx"

```
public class ColaImpresion {


    private Documento[] documentos;

    private int front;

    private int rear;

    private int cantidad;


    // Constructor: crea una cola de capacidad fija

    public ColaImpresion(int capacidad) {

        if (capacidad <= 0) {

            throw new IllegalArgumentException(

                "La capacidad debe ser mayor que cero."

            );
```

```
    }

    documentos = new Documento[capacidad];
    front = 0;
    rear = -1;
    cantidad = 0;
}

// Agrega un documento al final de la cola
public void enqueue(Documento documento) {
    if (isFull()) {
        throw new IllegalStateException(
            "Overflow: la cola de impresión está llena."
        );
    }

    rear++;
    documentos[rear] = documento;
    cantidad++;

    System.out.println(
        "Documento enviado: " + documento
    );
}

// Retira e imprime el siguiente documento
```

```
public Documento dequeue() {
    if (isEmpty()) {
        throw new IllegalStateException(
            "Underflow: no hay documentos para imprimir."
        );
    }

    Documento documento = documentos[front];

    // Liberar la posición utilizada
    documentos[front] = null;

    // Avanzar el frente de la cola
    front++;
    cantidad--;

    System.out.println(
        "Imprimiendo: " + documento.getNombre()
        + " - " + documento.getPaginas() + " páginas"
    );

    return documento;
}

// Consulta el próximo documento sin retirarlo
public Documento peek() {
```

```
        if (isEmpty()) {
            throw new IllegalStateException(
                "La cola de impresión está vacía."
            );
        }

        return documentos[front];
    }

    // Comprueba si la cola está vacía
    public boolean isEmpty() {
        return cantidad == 0;
    }

    // Comprueba si se alcanzó el final del arreglo
    public boolean isFull() {
        return rear == documentos.length - 1;
    }

    // Devuelve la cantidad de documentos pendientes
    public int size() {
        return cantidad;
    }
}
```

=== ENVÍO DE DOCUMENTOS ===

Documento enviado: Tesis.pdf (50 páginas)

Documento enviado: Contrato.docx (5 páginas)

Documento enviado: Foto.png (1 páginas)

Documentos pendientes: 3

=== IMPRESIÓN ===

Imprimiendo: Tesis.pdf - 50 páginas

=== PRÓXIMO DOCUMENTO ===

Siguiente en la cola: Contrato.docx (5 páginas)

Documentos pendientes: 2

Ejercicio 9

Quiero implementar en Java 21 una cola simple sobre un arreglo fijo de tamaño 5 para demostrar de forma practica el problema clasico del desperdicio de posiciones en memoria. El programa tiene que hacer una serie de operaciones consecutivas de enqueue hasta llenarla, despues hacer un par de dequeue para sacar elementos del frente, y luego intentar hacer un enqueue nuevo. En la salida por consola debe imprimir el estado interno del arreglo y los indices front y rear, mostrando como quedan celdas vacias libres a la izquierda pero que el programa dice que la cola esta llena porque rear llego al tope del vector, resultando en un falso desbordamiento.

Paso 1 (Llenado inicial): Crear una cola simple de tamaño 3. Ejecutar enqueue(1), enqueue(2), enqueue(3). Mostrar el arreglo: [1, 2, 3], con front = 0 y rear = 2.

Paso 2 (Liberación de elementos al frente): Ejecutar dequeue() y dequeue() (se retiran el 1 y el 2). Mostrar estado: front = 2, rear = 2, tamaño real = 1 elemento. Las posiciones [0] y [1] quedaron libres en la memoria.

Paso 3 (Falso desbordamiento a comprobar): Intentar hacer enqueue(4).

- Resultado esperado en consola: El sistema debe rebotar la inserción e indicar que la cola está llena (rear == capacidad - 1), evidenciando que los casilleros libres del inicio no se pueden aprovechar en una cola lineal simple.

import java.util.Arrays;

```
public class ColaLineal {

    private int[] arreglo;
    private int front;
    private int rear;

    public ColaLineal(int capacidad) {
        arreglo = new int[capacidad];
        front = -1;
        rear = -1;
    }

    // Inserta un elemento al final de la cola
    public void enqueue(int elemento) {

        // Problema: solo comprobamos si rear llegó al final
        if (rear == arreglo.length - 1) {
            System.out.println(
                "ERROR: La cola esta llena. No se puede insertar "
                + elemento + "."
            );
            return;
        }

        // Si la cola estaba vacía, inicializamos front
```

```
    if (front == -1) {
        front = 0;
    }

    rear++;
    arreglo[rear] = elemento;

    System.out.println("Enqueue(" + elemento + ") realizado.");
}

// Retira el elemento del frente
public int dequeue() {

    if (front == -1) {
        System.out.println("ERROR: La cola esta vacia.");
        return -1;
    }

    int elemento = arreglo[front];

    // Si era el último elemento, la cola queda vacía
    if (front == rear) {
        front = -1;
        rear = -1;
    } else {
        front++;
```

```
    }

    return elemento;
}

// Muestra el estado interno de la cola
public void mostrarEstado() {
    System.out.println("Arreglo: " + Arrays.toString(arreglo));
    System.out.println("front = " + front);
    System.out.println("rear = " + rear);

    int cantidad = (front == -1) ? 0 : rear - front + 1;
    System.out.println("Cantidad real de elementos = " + cantidad);
    System.out.println();
}

public static void main(String[] args) {

    ColaLineal cola = new ColaLineal(3);

    System.out.println("=== PASO 1: LLENADO INICIAL ===");

    cola.enqueue(1);
    cola.enqueue(2);
    cola.enqueue(3);
```

```
        cola.mostrarEstado();

        System.out.println("=== PASO 2: LIBERAR EL FRENTE ===");

        System.out.println("Dequeue: " + cola.dequeue());
        System.out.println("Dequeue: " + cola.dequeue());

        cola.mostrarEstado();

        System.out.println("=== PASO 3: INTENTAR INSERTAR 4 ===");

        cola.enqueue(4);

        cola.mostrarEstado();
    }
}
```

```
=== PASO 1: LLENADO INICIAL ===
Enqueue(1) realizado.
Enqueue(2) realizado.
Enqueue(3) realizado.
Arreglo: [1, 2, 3]
front = 0
rear = 2
Cantidad real de elementos = 3
```

=== PASO 2: LIBERAR EL FRENTE ===

Dequeue: 1

Dequeue: 2

Arreglo: [1, 2, 3]

front = 2

rear = 2

Cantidad real de elementos = 1

=== PASO 3: INTENTAR INSERTAR 4 ===

ERROR: La cola esta llena. No se puede insertar 4.

Arreglo: [1, 2, 3]

front = 2

rear = 2

Cantidad real de elementos = 1

Ejercicio 10

Necesito implementar en Java 21 una cola circular sobre un arreglo de tamaño fijo que resuelva el problema del desperdicio de memoria reutilizando las posiciones liberadas al inicio. El operador modulo % sirve para que los indices front y rear den la vuelta al inicio cuando superan el limite del vector ((posicion + 1) % capacidad). Se debe tener un contador para poder saber si la cola esta llena. El codigo debe incluir los metodos enqueue, dequeue, front, isEmpty e isFull con manejo de casos limite. Probar encolar, desencolar para liberar espacio y volver a encolar mostrando que ahora si reutiliza las celdas del principio sin trabarse.

Paso 1 (Llenar y vaciar el frente): Crear una ColaCircular de capacidad 3. Hacer enqueue(10), enqueue(20), enqueue(30) (cola llena). Desencolar dos veces con dequeue() (10 y 20 salen, liberando los índices 0 y 1).

Paso 2 (Reutilización del índice 0 con %): Hacer enqueue(40) → el índice debe calcularse como (2 + 1) % 3 = 0, guardando el 40 en la posición inicial [0] sin tirar error.

Paso 3 (Llenado circular y extracción ordenada): Hacer enqueue(50) (se ubica en el índice 1). Intentar un nuevo enqueue(60) → debe detectar correctamente que está llena (size == 3

o punteros cruzados). Desencolar todo → debe salir en orden estricto de inserción: 30, luego 40 y finalmente 50.

```
import java.util.Arrays;

public class ColaCircular {

    private int[] arreglo;
    private int front;
    private int rear;
    private int size;
    private int capacidad;

    public ColaCircular(int capacidad) {
        this.capacidad = capacidad;
        arreglo = new int[capacidad];

        front = 0;
        rear = -1;
        size = 0;
    }

    // Indica si la cola esta vacia
    public boolean isEmpty() {
        return size == 0;
    }

    // Indica si la cola esta llena
    public boolean isFull() {
        return size == capacidad;
    }

    // Inserta un elemento al final de la cola
    public void enqueue(int elemento) {

        if (isFull()) {
            System.out.println(
                "ERROR: Cola llena. No se puede insertar " + elemento
            );
            return;
        }

        // Calculamos la siguiente posicion circular
        rear = (rear + 1) % capacidad;
```

```
    arreglo[rear] = elemento;
    size++;

    System.out.println(
        "Enqueue(" + elemento + ") -> indice " + rear
    );
}

// Retira y devuelve el elemento del frente
public int dequeue() {

    if (isEmpty()) {
        System.out.println("ERROR: Cola vacia.");
        return -1;
    }

    int elemento = arreglo[front];

    // Avanzamos el frente de forma circular
    front = (front + 1) % capacidad;

    size--;

    // Si quedo vacia, reiniciamos los indices
    if (size == 0) {
        front = 0;
        rear = -1;
    }

    return elemento;
}

// Consulta el primer elemento sin retirarlo
public int front() {

    if (isEmpty()) {
        System.out.println("ERROR: Cola vacia.");
        return -1;
    }

    return arreglo[front];
}

// Muestra el estado interno
public void mostrarEstado() {
    System.out.println(
        "Arreglo: " + Arrays.toString(arreglo)
    );
```

```
        System.out.println("front = " + front);
        System.out.println("rear = " + rear);
        System.out.println("size = " + size);
        System.out.println("Vacia: " + isEmpty());
        System.out.println("Llena: " + isFull());
        System.out.println();
    }

    public static void main(String[] args) {

        ColaCircular cola = new ColaCircular(3);

        System.out.println("=== PASO 1: LLENAR LA COLA ===");

        cola.enqueue(10);
        cola.enqueue(20);
        cola.enqueue(30);

        cola.mostrarEstado();

        System.out.println("=== PASO 2: LIBERAR DOS POSICIONES ===");

        System.out.println("Dequeue: " + cola.dequeue());
        System.out.println("Dequeue: " + cola.dequeue());

        cola.mostrarEstado();

        System.out.println("=== PASO 3: REUTILIZAR EL INICIO ===");

        cola.enqueue(40);
        cola.mostrarEstado();

        System.out.println("=== PASO 4: COMPLETAR LA COLA ===");

        cola.enqueue(50);
        cola.mostrarEstado();

        System.out.println("=== PASO 5: INTENTAR INSERTAR 60 ===");

        cola.enqueue(60);
        cola.mostrarEstado();

        System.out.println("=== PASO 6: EXTRAER TODO ===");

        while (!cola.isEmpty()) {
            System.out.println("Dequeue: " + cola.dequeue());
        }
```

```
        cola.mostrarEstado();
    }
}
```

```
Dequeue: 30
Dequeue: 40
Dequeue: 50
```